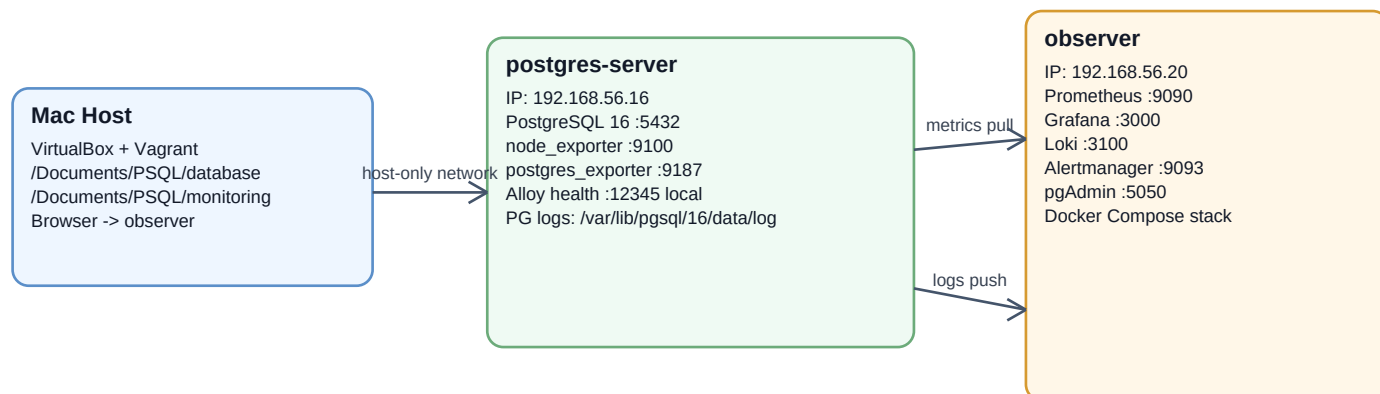


Mac PostgreSQL Monitoring Lab Runbook

GitHub-safe version. Placeholder passwords are used consistently: **pgadmin_password** for pgAdmin and **exporter_password** for the PostgreSQL exporter user.

1. Architecture

PostgreSQL Monitoring Lab Architecture



Prometheus scrapes postgres-server:9187 and :9100 every 15s.
Alloy tails PostgreSQL/systemd logs and pushes to Loki /loki/api/v1/push.
Grafana queries Prometheus for metrics and Loki for logs.

Host	IP	Runs	Role
postgres-server	192.168.56.16	PostgreSQL 16 node_exporter postgres_exporter Grafana Alloy	Database VM and log/metric source.
observer	192.168.56.20	Prometheus Grafana Loki Alertmanager pgAdmin	Observability and admin VM.

Flow	Path	Meaning
Metrics	Prometheus -> 192.168.56.16:9187 and :9100	Prometheus pulls PostgreSQL and OS metrics.
Logs	Alloy -> http://192.168.56.20:3100/loki/api/v1/push	Alloy tails PostgreSQL/systemd logs and pushes them to Loki.
Dashboards	Grafana -> Prometheus and Loki	Grafana displays metrics and logs; it does not store them.
Alerts	Prometheus -> Alertmanager	Prometheus evaluates alert rules and sends firing alerts.

2. Ports, URLs, And Credentials

Component	URL / endpoint	Username	Password / token
Prometheus	http://192.168.56.20:9090	-	-
Grafana	http://192.168.56.20:3000	admin	Admin@123
Loki health	http://192.168.56.20:3100/ready	-	-

Component	URL / endpoint	Username	Password / token
Alertmanager	http://192.168.56.20:9093	-	-
pgAdmin	http://192.168.56.20:5050	admin@example.com	pgadmin_password
PostgreSQL exporter DB user	postgres-server localhost:5432	exporter	exporter_password
PostgreSQL superuser	192.168.56.16:5432	postgres	Set your own before sharing

3. Health Checks

Service	Command / URL	Expected
PostgreSQL	systemctl is-active postgresql-16	active
node_exporter	curl -s http://localhost:9100/metrics head -3	Metrics text
postgres_exporter	curl -s http://localhost:9187/metrics grep '^pg_up'	pg_up 1
Alloy	curl -s http://127.0.0.1:12345/-/healthy	All Alloy components are healthy.
Prometheus targets	http://192.168.56.20:9090/targets	postgresql and node_exporter UP
Loki	curl http://192.168.56.20:3100/ready	ready
Docker stack	cd /root/monitoring && docker compose ps	All containers Up

4. Observability Stack Explanation

2. PROMETHEUS

Prometheus is the metrics collector.

It runs on:

observer

Container port:

9090

Browser URL:

<http://192.168.56.20:9090>

Important config file:

/root/monitoring/prometheus/prometheus.yml

Important config:

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'postgresql'
    static_configs:
      - targets: ['192.168.56.16:9187']

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['192.168.56.16:9100']
```

What this means:

Every 15 seconds, Prometheus connects to:

192.168.56.16:9187
postgres_exporter on postgres-server

192.168.56.16:9100
node_exporter on postgres-server

Prometheus pulls metrics from these endpoints and stores them locally inside the Prometheus Docker volume.

Prometheus is not reading PostgreSQL log files.

Prometheus is not pushing data.

Prometheus scrapes/pulls metrics from HTTP /metrics endpoints.

Examples of metrics:

```
pg_up
pg_stat_activity_count
node_cpu_seconds_total
node_memory_*
node_filesystem_*
```

Useful URLs:

<http://192.168.56.20:9090/targets>
Shows whether scrape targets are UP or DOWN.

<http://192.168.56.20:9090/query>
Prometheus query UI.

Useful Prometheus queries:

```
up
pg_up
```

3. POSTGRES_EXPORTER

postgres_exporter runs on:

postgres-server

Port:

```

9187
Metrics URL:

http://192.168.56.16:9187/metrics

Config file:

/etc/postgres_exporter.env

Config:

DATA_SOURCE_NAME="postgresql://exporter:exporter_password@localhost:5432/postgres?sslmode=disable"

What this means:

postgres_exporter logs into the local PostgreSQL database using:

user: exporter
password: exporter_password
database: postgres
host: localhost
port: 5432

It reads PostgreSQL status/statistics views and exposes them as
Prometheus metrics on port 9187.

The exporter user was created with:

CREATE USER exporter WITH PASSWORD 'exporter_password';
GRANT pg_monitor TO exporter;

Why pg_monitor:

pg_monitor lets the exporter read PostgreSQL monitoring views without
using a superuser account.

Health check:

curl -s http://localhost:9187/metrics | grep '^pg_up'

Expected:

pg_up 1

=====
4. NODE_EXPORTER
=====

node_exporter runs on:

postgres-server

Port:

9100

Metrics URL:

http://192.168.56.16:9100/metrics

What it does:

node_exporter exposes Linux operating system metrics from postgres-server.

Examples:

CPU
memory
disk
filesystem
network
load average

Prometheus scrapes node_exporter using this target:

192.168.56.16:9100

Health check:

curl -s http://localhost:9100/metrics | head -3

=====
5. LOKI
=====

```

Loki is the log storage and log query engine.

It runs on:

observer

Container port:

3100

Health URL:

<http://192.168.56.20:3100/ready>

Important point:

Loki does not scrape PostgreSQL logs by itself.

In this lab:

Alloy reads logs on postgres-server.

Alloy pushes those logs to Loki.

Loki stores them and makes them queryable.

Loki push endpoint:

<http://192.168.56.20:3100/loki/api/v1/push>

This endpoint is used by Alloy.

You do not manually browse this URL in Chrome.

Why <http://192.168.56.20:3100> shows 404:

Loki does not serve a homepage at /.

A 404 at / is normal.

Use /ready for health.

Useful Loki checks:

`curl http://192.168.56.20:3100/ready`

`curl "http://192.168.56.20:3100/loki/api/v1/labels"`

Expected labels after Alloy is working:

```
filename
host
job
level
service_name
unit
```

Query PostgreSQL logs:

```
curl -G "http://192.168.56.20:3100/loki/api/v1/query_range" \
--data-urlencode 'query={job="postgresql"}' \
--data-urlencode 'limit=20' \
--data-urlencode 'since=30m'
```

Query systemd logs:

```
curl -G "http://192.168.56.20:3100/loki/api/v1/query_range" \
--data-urlencode 'query={job="systemd"}' \
--data-urlencode 'limit=20' \
--data-urlencode 'since=30m'
```

=====

6. GRAFANA ALLOY

=====

Grafana Alloy runs on:

postgres-server

Local health port:

12345

Health check:

`curl -s http://127.0.0.1:12345/-/healthy`

Expected:

All Alloy components are healthy.

Config file:

```
/etc/alloy/config.alloy
```

What Alloy does in this lab:

1. Reads PostgreSQL log files.
2. Reads systemd journal logs.
3. Adds useful labels such as job and host.
4. Pushes logs to Loki on observer.

PostgreSQL log source block:

```
loki.source.file "postgresql" {
  targets = [{
    __path__ = "/var/lib/pgsql/16/data/log/postgresql-*.log",
    job      = "postgresql",
    host     = "postgres-server",
  }]

  forward_to = [loki.write.monitoring.receiver]
  tail_from_end = true

  file_match {
    enabled = true
    sync_period = "10s"
  }
}
```

What this means:

Alloy reads files matching:

```
/var/lib/pgsql/16/data/log/postgresql-*.log
```

It labels those log lines as:

```
job="postgresql"
host="postgres-server"
```

It forwards the logs to loki.write.monitoring.

Systemd journal source block:

```
loki.source.journal "systemd" {
  forward_to = [loki.write.monitoring.receiver]
  relabel_rules = loki.relabel.journal.rules
  max_age      = "12h"
  labels       = { job = "systemd", host = "postgres-server" }
}
```

What this means:

Alloy reads systemd journal logs from postgres-server.

It sends them to Loki.

It labels them as:

```
job="systemd"
host="postgres-server"
```

Loki destination block:

```
loki.write "monitoring" {
  endpoint {
    url = "http://192.168.56.20:3100/loki/api/v1/push"
  }
}
```

What this means:

All log entries collected by Alloy are pushed to Loki on observer.

Important permission note:

PostgreSQL log files are owned by postgres and may be mode 600.
Alloy needs ACL permissions to read them.

Working ACL commands:

```
setfacl -m u:alloy:--x /var/lib/pgsql /var/lib/pgsql/16 /var/lib/pgsql/16/data
setfacl -m u:alloy:r-x /var/lib/pgsql/16/data/log
setfacl -d -m u:alloy:r-x /var/lib/pgsql/16/data/log
setfacl -m u:alloy:r /var/lib/pgsql/16/data/log/postgresql-*.log
setfacl -d -m u:alloy:r /var/lib/pgsql/16/data/log
```

=====

7. GRAFANA

Grafana is the UI layer.

It runs on:

observer

Container port:

3000

Browser URL:

http://192.168.56.20:3000

Login:

user: admin
password: Admin@123

Grafana does not store the PostgreSQL metrics or logs itself.

Grafana queries:

Prometheus for metrics.
Loki for logs.

Inside Grafana, configure datasources as:

Prometheus:
http://prometheus:9090

Loki:
http://loki:3100

Why these URLs are different:

From Mac browser:

Use observer IP:
http://192.168.56.20:3000
http://192.168.56.20:9090
http://192.168.56.20:3100/ready

From inside Grafana container:

Use Docker Compose service names:
http://prometheus:9090
http://loki:3100

Grafana Explore test queries:

Prometheus datasource:

up
pg_up

Loki datasource:

{job="postgresql"}
{job="systemd"}

8. ALERTMANAGER

Alertmanager runs on:

observer

Container port:

9093

Browser URL:

http://192.168.56.20:9093

What it does:

Prometheus evaluates alert rules.
When a rule fires, Prometheus sends the alert to Alertmanager.
Alertmanager displays and manages the alert state.

Prometheus alerting config:

```
alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - alertmanager:9093
```

What this means:

Prometheus sends alerts to the alertmanager Docker service on port 9093.

Current alert rules:

```
PostgreSQLDown:
  expr: pg_up == 0

InstanceDown:
  expr: up == 0
```

```
=====
9. PGADMIN
=====
```

pgAdmin runs on:

observer

Container port:

5050 mapped to container port 80

Browser URL:

<http://192.168.56.20:5050>

pgAdmin web login:

```
email: admin@example.com
password: pgadmin_password
```

To register PostgreSQL server inside pgAdmin:

```
Name: postgres-server
Host: 192.168.56.16
Port: 5432
Maintenance database: postgres
Username: postgres
Password: the password you set for postgres
```

Important:

The docs did not set a postgres superuser password for the VM-based postgres-server lab. Set it manually if you want to connect as postgres:

```
sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'postgres_secret';"
```

```
=====
```


5. Clean Commands

PSQL MONITORING LAB - CLEAN COMMANDS ONLY

File: /Users/rrsolomo/Documents/PSQL/psql_monitoring_lab_commands.txt

```
=====
1. MAC HOST - CREATE DIRECTORIES AND START VMS
=====
```

```
cd /Users/rrsolomo/Documents/PSQL
mkdir -p database monitoring

cd /Users/rrsolomo/Documents/PSQL/database
vagrant up
vagrant ssh

cd /Users/rrsolomo/Documents/PSQL/monitoring
vagrant up
vagrant ssh
```

```
=====
2. MAC HOST - DATABASE VAGRANTFILE CONTENT
=====
```

```
cd /Users/rrsolomo/Documents/PSQL/database

cat > Vagrantfile <<'EOF'
Vagrant.configure("2") do |config|
  config.vm.box = "bento/almalinux-9"
  config.vm.box_architecture = "arm64"
  config.vm.hostname = "postgres-server"
  config.vm.network "private_network", ip: "192.168.56.16"

  config.vm.provider "virtualbox" do |vb|
    vb.name = "postgres-server"
    vb.cpus = 4
    vb.memory = 8192
  end
end
EOF
```

```
=====
3. MAC HOST - OBSERVER VAGRANTFILE CONTENT
=====
```

```
cd /Users/rrsolomo/Documents/PSQL/monitoring

cat > Vagrantfile <<'EOF'
Vagrant.configure("2") do |config|
  config.vm.box = "bento/almalinux-9"
  config.vm.box_architecture = "arm64"
  config.vm.hostname = "observer"
  config.vm.network "private_network", ip: "192.168.56.20"

  config.vm.provider "virtualbox" do |vb|
    vb.name = "observer"
    vb.cpus = 4
    vb.memory = 8192
  end
end
EOF
```

```
=====
4. POSTGRES-SERVER - INSTALL POSTGRESQL 16
=====
```

```
sudo -i

dnf install -y https://download.postgresql.org/pub/repos/yum/reporepms/EL-9-aarch64/pgdg-redhat-repo-
latest.noarch.rpm
dnf -qy module disable postgresql
dnf install -y postgresql16-server postgresql16-contrib

/usr/pgsql-16/bin/postgresql-16-setup initdb
systemctl enable --now postgresql-16

sudo -u postgres psql -c "SELECT version();"
sudo -u postgres psql -c "SHOW data_directory;"
```

```
=====
5. POSTGRES-SERVER - ENABLE REMOTE ACCESS
=====
```

```
PGDATA=/var/lib/pgsql/16/data
```

```
echo "listen_addresses = '*' >> "$PGDATA/postgresql.conf"
echo "host all all 0.0.0.0/0 scram-sha-256" >> "$PGDATA/pg_hba.conf"

systemctl reload postgresql-16
firewall-cmd --permanent --add-port=5432/tcp
firewall-cmd --reload
```

6. POSTGRES-SERVER - ENABLE POSTGRESQL LOGGING

```
sudo -u postgres psql <<'SQL'
ALTER SYSTEM SET logging_collector = 'on';
ALTER SYSTEM SET log_directory = 'log';
ALTER SYSTEM SET log_filename = 'postgresql-%a.log';
ALTER SYSTEM SET log_min_messages = 'warning';
SELECT pg_reload_conf();
SQL
```

```
systemctl restart postgresql-16
```

7. POSTGRES-SERVER - CREATE POSTGRES_EXPORTER USER

```
sudo -u postgres psql <<'SQL'
CREATE USER exporter WITH PASSWORD 'exporter_password';
GRANT pg_monitor TO exporter;
SQL
```

8. POSTGRES-SERVER - INSTALL NODE_EXPORTER

```
VER=1.8.2
curl -fsSL -o /tmp/node_exporter.tar.gz "https://github.com/prometheus/node_exporter/releases/download/v${VER}/node_exporter-${VER}.linux-arm64.tar.gz"
```

```
tar xzf /tmp/node_exporter.tar.gz -C /tmp
cp /tmp/node_exporter-${VER}.linux-arm64/node_exporter /usr/local/bin/
chmod +x /usr/local/bin/node_exporter
```

```
cat > /etc/systemd/system/node_exporter.service <<'EOF'
[Unit]
Description=Prometheus Node Exporter
After=network.target
```

```
[Service]
User=root
ExecStart=/usr/local/bin/node_exporter
Restart=always
```

```
[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl daemon-reload
systemctl enable --now node_exporter
firewall-cmd --permanent --add-port=9100/tcp
firewall-cmd --reload
curl -s http://localhost:9100/metrics | head -3
```

9. POSTGRES-SERVER - INSTALL POSTGRES_EXPORTER

```
VER=0.18.1
curl -fsSL -o /tmp/postgres_exporter.tar.gz "https://github.com/prometheus-community/postgres_exporter/releases/download/v${VER}/postgres_exporter-${VER}.linux-arm64.tar.gz"
```

```
tar xzf /tmp/postgres_exporter.tar.gz -C /tmp
cp /tmp/postgres_exporter-${VER}.linux-arm64/postgres_exporter /usr/local/bin/
chmod +x /usr/local/bin/postgres_exporter
```

```
cat > /etc/postgres_exporter.env <<'EOF'
DATA_SOURCE_NAME="postgresql://exporter:exporter_password@localhost:5432/postgres?sslmode=disable"
EOF
```

```
chmod 600 /etc/postgres_exporter.env
```

```
cat > /etc/systemd/system/postgres_exporter.service <<'EOF'
[Unit]
Description=Prometheus PostgreSQL Exporter
After=network.target
```

```
[Service]
```

```

User=postgres
Group=postgres
EnvironmentFile=/etc/postgres_exporter.env
ExecStart=/usr/local/bin/postgres_exporter
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now postgres_exporter
firewall-cmd --permanent --add-port=9187/tcp
firewall-cmd --reload
curl -s http://localhost:9187/metrics | grep '^pg_up'

=====
10. POSTGRES-SERVER - INSTALL GRAFANA ALLOY
=====

dnf install -y https://github.com/grafana/alloy/releases/download/v1.17.1/alloy-1.17.1-1.arm64.rpm
alloy --version

cat > /etc/alloy/config.alloy <<'EOF'
// PostgreSQL log files
loki.source.file "postgresql" {
    targets = [{
        __path__ = "/var/lib/pgsql/16/data/log/postgresql-*.log",
        job      = "postgresql",
        host     = "postgres-server",
    }]

    forward_to = [loki.write.monitoring.receiver]
    tail_from_end = true

    file_match {
        enabled = true
        sync_period = "10s"
    }
}

// systemd journal labels
loki.relabel "journal" {
    forward_to = []

    rule {
        source_labels = ["__journal__systemd_unit"]
        target_label = "unit"
    }

    rule {
        source_labels = ["__journal_priority_keyword"]
        target_label = "level"
    }
}

// systemd journal
loki.source.journal "systemd" {
    forward_to = [loki.write.monitoring.receiver]
    relabel_rules = loki.relabel.journal.rules
    max_age = "12h"
    labels = { job = "systemd", host = "postgres-server" }
}

// Loki endpoint on observer
loki.write "monitoring" {
    endpoint {
        url = "http://192.168.56.20:3100/loki/api/v1/push"
    }
}
EOF

usermod -aG adm,systemd-journal,postgres alloy

setfacl -m u:alloy:--x /var/lib/pgsql /var/lib/pgsql/16 /var/lib/pgsql/16/data
setfacl -m u:alloy:r-x /var/lib/pgsql/16/data/log
setfacl -d -m u:alloy:r-x /var/lib/pgsql/16/data/log
setfacl -m u:alloy:r /var/lib/pgsql/16/data/log/postgresql-*.log
setfacl -d -m u:alloy:r /var/lib/pgsql/16/data/log

sudo -u alloy ls /var/lib/pgsql/16/data/log/
alloy fmt /etc/alloy/config.alloy
systemctl reset-failed alloy
systemctl enable --now alloy
systemctl restart alloy

```

```
curl -s http://127.0.0.1:12345/-/healthy
```

```
=====
11. OBSERVER - INSTALL DOCKER
=====
```

```
sudo -i
```

```
dnf install -y dnf-plugins-core
dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
dnf install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
systemctl enable --now docker
```

```
docker --version
docker compose version
```

```
mkdir -p /root/monitoring/prometheus/rules
mkdir -p /root/monitoring/loki
mkdir -p /root/monitoring/alertmanager
mkdir -p /root/monitoring/grafana/provisioning/datasources
cd /root/monitoring
```

```
=====
12. OBSERVER - CREATE ENV FILE
=====
```

```
cat > /root/monitoring/.env <<'EOF'
DB_HOST=192.168.56.16
DB_STANZA=prod01
GRAFANA_ADMIN_PASSWORD=Admin@123
PGADMIN_EMAIL=admin@example.com
PGADMIN_PASSWORD=pgadmin_password
EOF
```

```
=====
13. OBSERVER - CREATE PROMETHEUS CONFIG
=====
```

```
cat > /root/monitoring/prometheus/prometheus.yml <<'EOF'
global:
  scrape_interval: 15s
  evaluation_interval: 15s

alerting:
  alertmanagers:
    - static_configs:
      - targets:
        - alertmanager:9093

rule_files:
  - /etc/prometheus/rules/*.yml

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['prometheus:9090']

  - job_name: 'postgresql'
    static_configs:
      - targets: ['192.168.56.16:9187']

  - job_name: 'node_exporter'
    static_configs:
      - targets: ['192.168.56.16:9100']
EOF
```

```
=====
14. OBSERVER - CREATE ALERT RULES
=====
```

```
cat > /root/monitoring/prometheus/rules/alerts.yml <<'EOF'
groups:
  - name: postgresql_alerts
    rules:
      - alert: PostgreSQLDown
        expr: pg_up == 0
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "PostgreSQL is down on {{ $labels.instance }}"

  - name: node_alerts
    rules:
      - alert: InstanceDown
        expr: up == 0
EOF
```

```

        for: 2m
        labels:
            severity: critical
        annotations:
            summary: "Target {{ $labels.job }} down on {{ $labels.instance }}"
EOF

=====
15. OBSERVER - CREATE DOCKER COMPOSE FILE
=====

cat > /root/monitoring/docker-compose.yml <<'EOF'
services:
    prometheus:
        image: prom/prometheus:latest
        container_name: prometheus
        restart: unless-stopped
        ports:
            - "9090:9090"
        volumes:
            - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
            - ./prometheus/rules:/etc/prometheus/rules:ro
            - prometheus_data:/prometheus

    grafana:
        image: grafana/grafana:latest
        container_name: grafana
        restart: unless-stopped
        ports:
            - "3000:3000"
        environment:
            GF_SECURITY_ADMIN_USER: admin
            GF_SECURITY_ADMIN_PASSWORD: ${GRAFANA_ADMIN_PASSWORD:-Admin@123}
        volumes:
            - grafana_data:/var/lib/grafana

    loki:
        image: grafana/loki:latest
        container_name: loki
        restart: unless-stopped
        ports:
            - "3100:3100"

    alertmanager:
        image: prom/alertmanager:latest
        container_name: alertmanager
        restart: unless-stopped
        ports:
            - "9093:9093"

    pgadmin:
        image: dpage/pgadmin4:latest
        container_name: pgadmin
        restart: unless-stopped
        ports:
            - "5050:80"
        environment:
            PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL:-admin@example.com}
            PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD:-pgadmin_password}
        volumes:
            - pgadmin_data:/var/lib/pgadmin

volumes:
    prometheus_data:
    grafana_data:
    pgadmin_data:
EOF

cd /root/monitoring
docker compose up -d
docker compose ps

=====
16. OPTIONAL - SET POSTGRES SUPERUSER PASSWORD FOR PGADMIN
=====

sudo -u postgres psql -c "ALTER USER postgres WITH PASSWORD 'postgres_secret';"

=====
17. POSTGRES-SERVER HEALTH CHECKS
=====

systemctl is-active postgresql-16 node_exporter postgres_exporter alloy
curl -s http://localhost:9100/metrics | head -3
curl -s http://localhost:9187/metrics | grep '^pg_up'
curl -s http://127.0.0.1:12345/-/healthy

```

```
sudo -u postgres psql -c "DO \$$ BEGIN RAISE WARNING 'loki test from postgres-server'; END \$$;"
```

18. OBSERVER HEALTH CHECKS

```
cd /root/monitoring
docker compose ps

curl -s http://localhost:9093/-/healthy
curl -s http://localhost:3100/ready
curl -s "http://localhost:3100/loki/api/v1/labels"

curl -G "http://192.168.56.20:3100/loki/api/v1/query_range" \
  --data-urlencode 'query={job="postgresql"}' \
  --data-urlencode 'limit=20' \
  --data-urlencode 'since=30m'

curl -G "http://192.168.56.20:3100/loki/api/v1/query_range" \
  --data-urlencode 'query={job="systemd"}' \
  --data-urlencode 'limit=20' \
  --data-urlencode 'since=30m'
```

19. BROWSER URLS

```
Prometheus:
http://192.168.56.20:9090

Prometheus targets:
http://192.168.56.20:9090/targets

Grafana:
http://192.168.56.20:3000

Alertmanager:
http://192.168.56.20:9093

Loki health:
http://192.168.56.20:3100/ready

pgAdmin:
http://192.168.56.20:5050
```

20. GRAFANA DATASOURCE VALUES

```
Prometheus datasource URL:
http://prometheus:9090

Loki datasource URL:
http://loki:3100
```

21. GRAFANA EXPLORE TEST QUERIES

```
up

pg_up

{job="postgresql"}

{job="systemd"}
```

22. PGADMIN LOGIN AND SERVER REGISTRATION VALUES

```
pgAdmin URL:
http://192.168.56.20:5050

pgAdmin email:
admin@example.com

pgAdmin password:
pgadmin_password

pgAdmin server registration:
Name: postgres-server
Host: 192.168.56.16
Port: 5432
Maintenance database: postgres
Username: postgres
```

Password: postgres_secret